



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MEXICO

FACULTAD DE INGENIERÍA

Materia:

BASES DE DATOS II

"Documentación técnica".

Profesor: Pablo Salas Castillo

Grupo: CO02

Índice.

1.	<u>Descripción del problema.....</u>	<u>2</u>
2.	<u>Propuesta de solución.....</u>	<u>3</u>
3.	<u>Requerimientos de hardware</u>	<u>4</u>
4.	<u>Requerimientos de software.....</u>	<u>5</u>
5.	<u>Ciclo de vida.....</u>	<u>6</u>
6.	<u>Codificación.....</u>	<u>10</u>
7.	<u>Evidencias.....</u>	<u>11</u>
8.	<u>Referencias bibliográficas.....</u>	<u>13</u>

Descripción del problema.

La cadena de tiendas tipo OfficeMax enfrenta diversos problemas relacionados con el manejo y control de la información en sus sucursales, ya que cada una opera de forma independiente y administra por separado procesos importantes como inventarios, ventas, proveedores, precios y atención al cliente, lo que provoca duplicación de datos, retrasos en la toma de decisiones y muchas dificultades para generar reportes consolidados, además esta falta de integración impide tener una visión completa del negocio y aumenta la probabilidad de errores debido a que cada tienda mantiene su propia información sin un mecanismo de sincronización adecuado, lo que complica la coordinación entre sucursales y genera inconsistencias que afectan la precisión de los registros y reducen la confiabilidad de la información disponible.

Cuando un cliente solicita saber si un producto está disponible en otra tienda, el proceso se vuelve lento porque la comunicación entre sucursales se realiza de forma manual o telefónica, lo que provoca tiempos de espera innecesarios y una atención menos eficiente, este sistema fragmentado funciona como un cuello de botella que afecta las operaciones diarias y reduce la calidad del servicio, además, el sistema actual no está preparado para manejar bases de datos distribuidas, lo que es esencial para asegurar la continuidad del negocio en caso de fallos locales y mejorar la velocidad de acceso a la información en distintas zonas, por lo que cualquier interrupción en una sucursal puede impactar directamente en el rendimiento de toda la cadena.

La falta de control centralizado también dificulta la implementación de estrategias comerciales globales como promociones, cambios de precio o actualizaciones de catálogos, lo que provoca que cada tienda funcione casi como una unidad independiente, esta situación incrementa el riesgo de errores humanos y genera un uso ineficiente del tiempo del personal, ya que muchas actividades que podrían ser automáticas deben hacerse manualmente, además el seguimiento de clientes y la facturación no están estandarizados, lo que complica rastrear compras, quejas o solicitudes, afectando la calidad del servicio y reduciendo las oportunidades de fidelización.

Ante estos problemas, la empresa necesita un sistema de información moderno, seguro y flexible que integre de manera centralizada los procesos esenciales de todas las sucursales, permitiendo acceso a datos en tiempo real y reduciendo los

errores que actualmente se generan por la falta de sincronía, este sistema debe contar con una base de datos distribuida en dos sitios que trabajen de manera conjunta para asegurar disponibilidad continua, de esta forma se logrará una operación más eficiente, decisiones más precisas y una experiencia de cliente más rápida y satisfactoria, lo que contribuirá tanto al control interno como al fortalecimiento de la competitividad de la cadena en el mercado actual.

PROPUESTA DE SOLUCIÓN.

Para resolver los problemas actuales de la cadena de tiendas se propone el diseño e implementación de un sistema de información centralizado que integre todos los procesos principales como ventas, inventarios, proveedores, precios y control de clientes, este sistema permitirá que cada sucursal trabaje con datos sincronizados y actualizados en tiempo real, eliminando la duplicación de información y mejorando la eficiencia en la operación diaria, además se plantea el uso de una base de datos distribuida en dos sitios que funcionarán como nodos coordinados, permitiendo que la empresa mantenga disponibilidad continua aunque uno de los nodos falle, lo que asegura mayor estabilidad y continuidad del negocio.

El sistema trabajará mediante una arquitectura cliente servidor donde cada sucursal tendrá acceso a la aplicación a través de una red interna y una conexión segura hacia los servidores de datos, permitiendo que las operaciones comunes como registrar ventas, consultar existencias o revisar información de proveedores se realicen de forma rápida y sin depender de procesos manuales, también se incluirá un módulo de reportes que consolide la información de todas las sucursales, permitiendo una mejor toma de decisiones y ofreciendo una visión clara de la empresa en tiempo real.

La base de datos distribuida se configurará bajo un esquema maestro réplica o réplica dual según las necesidades del proyecto, asegurando sincronización constante entre los dos sitios y disminuyendo el riesgo de pérdida de información, además se establecerán mecanismos de respaldo programado y recuperación ante fallos para garantizar que el sistema mantenga su funcionamiento incluso ante interrupciones imprevistas, con esto las tiendas podrán consultar información de forma más rápida, actualizar precios de manera centralizada y mejorar la coordinación entre sucursales.

La propuesta también incluye una interfaz intuitiva y sencilla para que los empleados puedan operar el sistema sin complicaciones, esto reducirá errores humanos y permitirá automatizar tareas repetitivas como actualización de inventarios, registro de movimientos y emisión de facturas, de esta manera se logrará un flujo de trabajo más ágil, organizado y seguro, apoyando el crecimiento de la cadena y mejorando la experiencia del cliente al recibir un servicio más rápido y eficiente.

REQUERIMIENTOS DE HARDWARE.

1. Servidores principales con buena capacidad de procesamiento y memoria, el sistema necesita manejar información de varias sucursales al mismo tiempo, se requiere que cada servidor tenga un procesador multinúcleo y entre 8 y 16 GB de RAM, permitirá que las consultas, registros de ventas, actualizaciones de inventario y sincronización entre nodos se realicen de manera rápida, evitando que el sistema se trabe cuando varias tiendas lo están usando al mismo tiempo
2. Almacenamiento sólido tipo SSD con suficiente espacio, se recomienda un mínimo de 512 GB por servidor porque este tipo de unidades permite que la lectura y escritura de datos sea mucho más rápida, lo cual es importante para que la replicación entre los dos sitios distribuidos se mantenga al día y no haya retrasos en la información que consulten las sucursales, además el espacio permitirá guardar historiales y respaldos locales
3. Conexiones de red de alta velocidad en los servidores, una conexión de al menos 1 Gbps es necesaria para que la comunicación entre los dos nodos de la base de datos sea constante y no genere fallas por desactualización, esto es importante porque si un nodo se queda atrás la información se puede volver inconsistente, así que una red rápida ayuda a mantener la sincronía entre los sitios
4. Equipos de escritorio o terminales para las sucursales, cada tienda necesitará computadoras con características básicas pero suficientes para correr la aplicación del sistema sin lentitud, se recomienda que tengan procesadores de gama media y entre 4 y 8 GB de RAM, así los empleados podrán registrar ventas, consultar inventarios y atender a los clientes sin que el sistema tarda en cargar o se quede congelado
5. Dispositivos de respaldo físico, como discos duros externos o almacenamiento en red, esto permitirá guardar copias de seguridad

periódicas para evitar la pérdida total de información en caso de apagones, errores humanos o fallos en los servidores, estas copias de respaldo son una parte importante del sistema ya que ayudan a recuperar los datos en caso de un problema grave

6. Una red local estable dentro de cada sucursal, esto implica contar con routers, switches y cableado adecuado que permita que todas las estaciones de trabajo se conecten entre sí y al servidor central, una buena red interna evita desconexiones inesperadas y asegura que la comunicación con el sistema sea continua

REQUERIMIENTOS DE SOFTWARE.

1. Un sistema gestor de base de datos que permita replicación distribuida, se recomienda usar MySQL, MariaDB o PostgreSQL, ya que estos motores soportan la creación de nodos distribuidos y permiten que la información se sincronice entre dos sitios diferentes, esta característica es clave para asegurar que las sucursales siempre trabajen con datos actualizados y para que el sistema siga funcionando incluso si uno de los nodos llega a fallar
2. Un sistema operativo estable para servidores, como Linux Ubuntu Server o Windows Server, estos sistemas permiten una administración más segura del hardware, mejoran el rendimiento de los procesos internos y ofrecen herramientas para monitorear el estado del servidor, además son compatibles con la mayoría de los motores de base de datos y frameworks de desarrollo, lo que facilita la implementación del sistema
3. Lenguajes de programación adecuados para el desarrollo, como Java, C#, Python o PHP, estos lenguajes son ampliamente usados en el desarrollo de sistemas empresariales porque permiten crear aplicaciones robustas, escalables y fáciles de mantener, además cuentan con bibliotecas y herramientas que facilitan la creación de módulos como ventas, inventarios, reportes y administración de usuarios
4. Frameworks o plataformas de desarrollo, como .NET, Laravel, Django o JavaFX, estos frameworks ayudan a crear interfaces más amigables, organizan el código y facilitan la conexión entre la aplicación y la base de datos, permitiendo que el sistema sea más estable y seguro, además reducen el tiempo de desarrollo ya que incluyen muchas funciones listas para usar

5. Herramientas de diseño y documentación, estas herramientas permiten crear diagramas de clases, diagramas de flujo, diagramas entidad relación y toda la documentación técnica necesaria para entender el funcionamiento interno del sistema, esto ayuda a organizar las ideas del proyecto y garantiza que cualquier persona pueda comprender cómo está construido
6. Plataformas de control de versiones, como Git junto con GitHub o GitLab, estas herramientas permiten trabajar el proyecto de manera ordenada, ya que registran todos los cambios que se realizan en el sistema, permiten coordinar trabajo entre varios desarrolladores y facilitan regresar a una versión anterior en caso de que algún cambio provoque un error.
7. Software de seguridad, como firewall, VPN y certificados SSL, estos elementos son indispensables para proteger la información que viaja entre las sucursales y los servidores, ya que el sistema maneja datos importantes como precios, inventarios y posiblemente datos de clientes, por lo que es necesario asegurar que la comunicación sea cifrada y que no existan accesos no autorizados

Ciclo de Vida del Desarrollo de Sistemas de Bases de Datos.

El desarrollo de un sistema de bases de datos no es solo un proceso técnico; es un recorrido lleno de decisiones, revisiones y momentos donde las ideas se transforman en algo funcional y real. A lo largo de este ciclo, cada paso contribuye a dar forma a una solución que responda a necesidades concretas y que, con suerte, haga la vida un poco más fácil para quienes usarán el sistema. A continuación, se describen los pasos fundamentales de este ciclo, cada uno con su propio propósito y su toque esencial dentro del proyecto.

1. Análisis de Requerimientos

Todo comienza aquí. En este punto, el objetivo es entender qué necesita realmente la organización.

Se escuchan problemas, se observan procesos, se toman notas y, poco a poco, se va dibujando el panorama completo. A veces surgen dudas; otras veces, nuevas ideas. Pero, sobre todo, surge claridad.

Durante esta etapa se identifican los datos que deben almacenarse, quién los utilizará y para qué. Es un momento clave porque cualquier malentendido aquí puede arrastrarse hacia adelante y complicar las cosas más tarde. Por eso, se conversa, se pregunta, se vuelve a preguntar... y se valida todo.

2. Diseño Conceptual

Con las necesidades bien comprendidas, ahora toca imaginar la estructura del sistema sin preocuparse todavía por lo técnico.

Es una fase creativa y bastante visual: entidades, relaciones, atributos... todo empieza a tomar forma como si se armara un rompecabezas.

Se elabora un modelo entidad–relación que representa el corazón del sistema. Aquí se siente un poco el alivio de ver cómo las ideas empiezan a ordenarse de manera lógica y comprensible.

3. Diseño Lógico

Después del diseño conceptual llega el momento de convertir esas ideas en algo más cercano a la realidad técnica.

Se elige el modelo lógico más adecuado —generalmente relacional— y se transforman las entidades, relaciones y atributos en tablas, claves y restricciones.

Es un paso más detallado. A veces requiere pequeños ajustes que no se habían visto antes, pero que ahora se vuelven evidentes. Se define cómo convivirán los datos y cómo se mantendrá su integridad. Poco a poco se siente como si el sistema comenzará a “respirar”.

4. Diseño Físico

Aquí se entra en terreno más técnico.

Se decide cómo se almacenarán los datos en el servidor, qué índices se usarán, qué motor de base de datos es el ideal y cómo optimizar el rendimiento. Esta parte es crucial porque un buen diseño físico puede marcar la diferencia entre un sistema que fluye... y uno que desespera.

Es una fase donde se piensa en tiempos de respuesta, en el volumen de datos y en cómo garantizar un acceso rápido y seguro. Es el momento de afinar detalles antes de pasar a la acción.

5. Implementación

Es cuando todo lo planeado finalmente toma forma dentro de un sistema real.

Se crean las bases de datos, las tablas, las vistas, los procedimientos y las reglas necesarias. También se desarrolla la aplicación que interactúa con la base de datos.

Esta fase está llena de pequeños logros: que la primera consulta funcione, que los datos se guarden correctamente, que los módulos empiecen a verse como una solución completa. También pueden aparecer errores inesperados, pero es parte del camino.

6. Pruebas

Con el sistema construido, llega el momento de ponerlo a prueba.

Se verifica que todo funcione como debe: que las consultas devuelvan lo correcto,

que los usuarios puedan acceder sin problemas, que la información se mantenga consistente y que los procesos no se detengan.

Es una etapa intensa. A veces aparecen fallos, pero cada error corregido hace que el sistema se sienta más sólido y confiable.

7. Implementación en Producción

Después de tanta planeación y correcciones, el sistema llega a su destino: el entorno real.

Aquí empieza a trabajar con datos verdaderos y usuarios reales. La migración puede generar un poco de nervios, pero también mucha satisfacción al ver el impacto final.

El sistema comienza a conectar procesos, mejorar tiempos, reducir errores... es donde todo el esfuerzo cobra sentido.

8. Mantenimiento y Mejora Continua

Aunque muchos creen que el proyecto termina cuando se entrega, la verdad es que aquí comienza una nueva etapa.

El sistema se monitorea, se ajusta, se corrigen detalles que aparecen con el uso diario y, cuando es necesario, se agregan nuevas funcionalidades.

El mantenimiento es una manera de cuidar y hacer crecer lo que se construyó. Es un recordatorio de que un sistema de información está vivo y debe evolucionar con la organización.

Codificación.

En este repositorio se fue guardando todo el código:

https://github.com/zdenc0de/proyecto_BDII/tree/modelo_fisico

Evidencias.





REFERENCIAS BIBLIOGRÁFICAS.

- https://github.com/zdenc0de/proyecto_BDII/tree/modelo_fisico
- <https://www.open.edu/openlearn/science-maths-technology/the-databases-e-development-life-cycle/content-section-1.2>
- <https://www.microsoft.com/en-ca/power-platform/topics/phases-of-the-software-development-lifecycle>
- <https://www.northware.mx/blog/requerimientos-en-el-desarrollo-de-software-y-aplicaciones/>